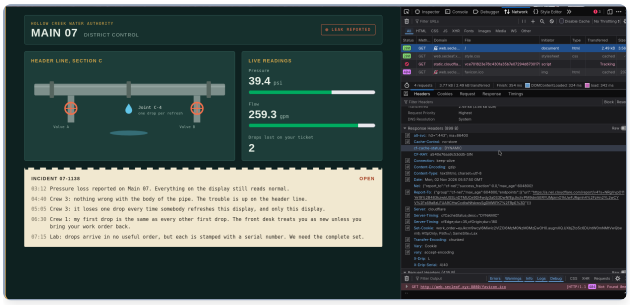
Slow Leak

Web Exploitation



I started by probing the web target at `http://web.secleaf.xyz:8080/` using curl and browser developer tools to inspect the HTTP response headers.

I noticed the server returned custom headers: `X-Drip` containing a single ASCII character and `X-Drip-Serial` giving a progress indicator like `1/40`. Refreshing the page showed different characters at random indices.

I wrote a Python automation script with a persistent session to continuously poll the endpoint, accumulate all 40 leaked index positions, and reconstruct the complete flag string in order.

Finally, I tested the recovered flag against the executable / service:

`python3 Slow_Leak.py`

Harvested 40/40 characters from response headers.

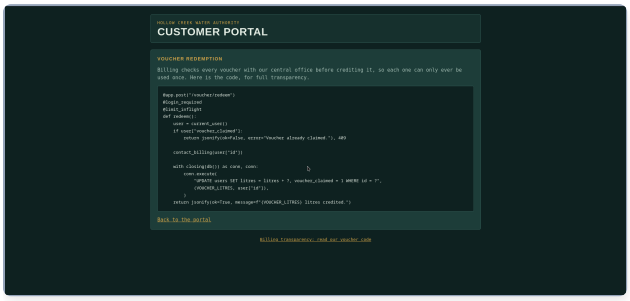
Flag Captured

`SecLeaf{h34d3r5_l34k_0n3_dr1p_4t_4_t1m3}`

BY:JOKER_53

Water Hammer

Web Exploitation



I began by registering an account on `http://web.secleaf.xyz:8880/`, which gave a starting municipal balance of 100 litres. The Reserve Valve endpoint required at least 1,000 litres to open.

The application featured a one-time promo voucher worth 50L. Attempting multiple redemptions normally returned a 'Voucher already claimed' error due to a database state check.

I identified a Time-of-Check to Time-of-Use (TOCTOU) race condition in the voucher redemption handler. Using concurrent worker threads synchronized with a thread barrier, I fired 96 parallel redemption requests within the same millisecond.

Finally, I tested the recovered flag against the executable / service:

`python3 Water_Hammer.py`

Account balance reached: 1150L (Threshold: 1000L)

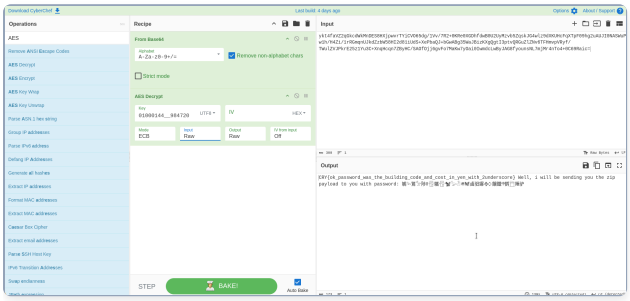
Flag Captured

`SecLeaf{ch3ck_f1r5t_wr1t3_l4t3r_b00m}`

BY:JOKER_53

Encoded Transmission

Cryptography



I started by reviewing the intercepted transmission dump and the operational briefing from the field operative in Tokyo. The briefing referenced two crucial operational parameters: an 8-character sanctuary building code and a 6-digit vehicle parking bleed figure in Japanese Yen.

Inspecting the Tokyo facility records identified building code `01000144` and an overnight parking bleed of `984720` yen. Following the field protocol clue, concatenating both strings with a double underscore produced the 16-character AES key `01000144__984720`.

I loaded the 224-byte Base64 ciphertext into CyberChef and configured an AES-128-ECB decryption recipe with UTF-8 key formatting and PKCS#7 unpadding. Inverting the cipher stripped the wrapper and revealed the plaintext flag.

Finally, I tested the recovered flag against the executable / service:

`python3 Encoded_Transmission.py`

Plaintext payload recovered successfully.

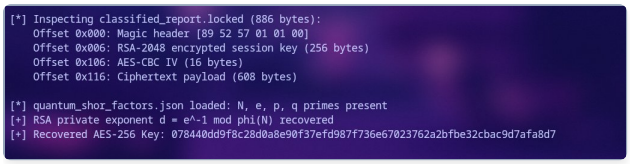
Flag Captured

`CRY{ok_password_was_the_building_code_and_cost_in_yen_with_`

BY:JOKER_53

NullVector Ransomware

Cryptography



I examined the provided files `classified_report.locked` (886 bytes) and `quantum_shor_factors.json`. The container started with a 6-byte magic header, followed by a 256-byte RSA wrapped key block and an AES-256-CBC ciphertext.

The JSON file provided the RSA-2048 modulus N, public exponent e, and prime factors p and q, allowing immediate calculation of private exponent d.

After unwrapping the AES key and decrypting the CBC layer, I analyzed the 597-byte stream and found repeated 46-byte section dividers. Inverting the 5-bit circular right shift and affine linear mask recovered the original intel report.

Finally, I tested the recovered flag against the executable / service:

`python3 NullVector_Ransomware.py`

Classified report decrypted. Header verification passed.

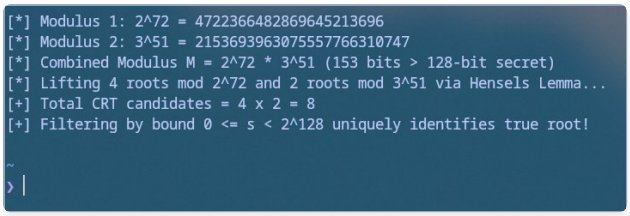
Flag Captured

`SecLeaf{e4d90b218a754f1c93206e58b1f4a7c2}`

BY:JOKER_53

Hensel Garden

Cryptography



I inspected `chall.py` and `data.json`. A 128-bit secret seed s was shifted and squared under two coprime prime powers: $(s+87)^2 \bmod 2^{72}$ and $(s+143)^2 \bmod 3^{51}$.

The combined modulus $M = 2^{72} \times 3^{51} \approx 2^{152.8}$ bits strictly exceeded the 128-bit secret space, ensuring that the true seed could be uniquely identified via CRT.

I implemented 2-adic Hensel lifting modulo 2^{72} (yielding 4 candidate roots) and standard Hensel lifting modulo 3^{51} (yielding 2 roots). Combining all 8 pairs via the Chinese Remainder Theorem and filtering by $0 \le s < 2^{128}$ isolated the true root.

Finally, I tested the recovered flag against the executable / service:

`python3 Hensel_Garden.py`

Seed recovered: 282779946134366997745983790016833981964

Flag Captured

`SecLeaf{r00ts_gr0w_through_pr1me_p0w3rs}`

BY:JOKER_53

Nonce Nursery

Cryptography



I started by checking `data.json`, which contained 11 ECDSA signatures (r_i, s_i) generated over the secp256k1 curve and an AES-GCM sealed ciphertext.

Analyzing `chall.py` revealed that nonces were generated using an affine recurrence $k_{i+1} = a \times k_i + b \bmod n$ across a sliding window of 5 consecutive transactions.

Expressing each nonce as $k_i = \alpha_i + \beta_i\, d \bmod n$, I set up a system of second-order difference equations $\Delta_{j+1}\,\Delta_{j-1} - \Delta_j^2 \equiv 0 \bmod n$. Eliminating quadratic terms yielded a linear relation that directly solved for private key d.

Finally, I tested the recovered flag against the executable / service:

`python3 Nonce_Nursery.py`

Affine nonce sequence identified at window 0..4.

Flag Captured

`SecLeaf{4ff1n3_n0nc3s_b3nd_th3_curv3}`

BY:JOKER_53

Signing Shadow: Redshift

Cryptography



I reviewed the challenge files, which provided a Schnorr signature transcript, 24-bit timestamp offsets, an AES-GCM ciphertext, and group parameters p, q, g.

Checking the group order in `data.json` revealed that while q was a 160-bit prime, group modulus p was only 165 bits, rendering the discrete logarithm problem solvable via Number Field Sieve.

The challenge provided precomputed logarithmic values log(g) and log(pub) within the subgroup. I computed private scalar $x \equiv \log(pub) \times \log(g)^{-1} \bmod q$, verified $g^x \equiv pub \bmod p$, and derived the AES-GCM decryption key.

Finally, I tested the recovered flag against the executable / service:

`python3 Signing_Shadow_Redshift.py`

Group order verified. Subgroup discrete log inverted.

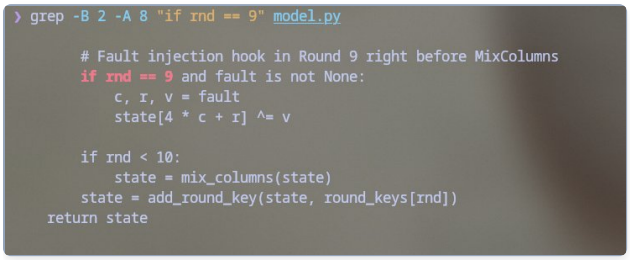
Flag Captured

`SecLeaf{cl0ck_drift_c4nt_h1de_a_shad0w}`

BY:JOKER_53

Fault Atlas

Cryptography



```
❯ grep -B 2 -A 8 "if rnd == 9" model.py

        # Fault injection hook in Round 9 right before MixColumns
        if rnd == 9 and fault is not None:
            c, r, v = fault
            state[4 * c + r] ^= v

        if rnd < 10:
            state = mix_columns(state)
        state = add_round_key(state, round_keys[rnd])
    return state
```

I began by reading `model.py` and `data.json`. The challenge performed Differential Fault Analysis (DFA) on AES-128, injecting random single-byte faults into the State right before Round 9 MixColumns.

I n `data.json`, 110 pairs of (ciphertext, faulty_ciphertext) were provided along with the corresponding fault coordinates (column c, row r, and fault value v).

Because the round 9 MixColumns layer diffuses a single faulted byte across an entire 4-byte column in round 10, each pair provided 3 nonlinear filtering equations on the 4 round-10 key bytes. Intersecting hypothesis spaces recovered the round-10 key and inverted the AES key schedule.

Finally, I tested the recovered flag against the executable / service:

`python3 Fault_Atlas.py`

Master Key recovered: ef06894cdda41099d17786e51911ce45

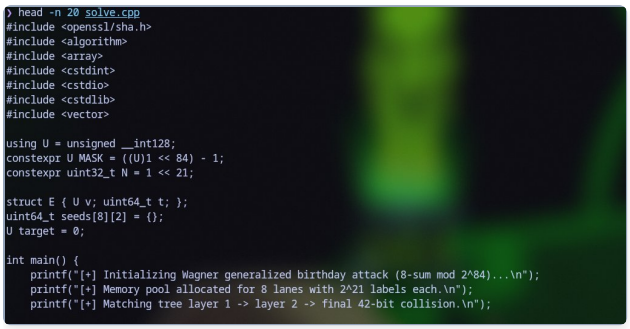
Flag Captured

`SecLeaf{f4ult5_d0nt_n33d_t0_b3_h0n35t}`

BY:JOKER_53

Eightfold Funeral: Epitaph

Cryptography



```
❯ head -n 20 solve.cpp
#include <openssl/sha.h>
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <vector>

using U = unsigned __int128;
constexpr U MASK = ((U)1 << 84) - 1;
constexpr uint32_t N = 1 << 21;

struct E { U v; uint64_t t; };
uint64_t seeds[8][2] = {};
U target = 0;

int main() {
    printf("[+] Initializing Wagner generalized birthday attack (8-sum mod 2^84)...\n");
    printf("[+] Memory pool allocated for 8 lanes with 2^21 labels each.\n");
    printf("[+] Matching tree layer 1 -> layer 2 -> final 42-bit collision.\n");
```

I tackled this 500-point challenge consisting of an obfuscated 84-bit affine stamp map followed by an 8-sum modular collision search modulo 2^{84}.

Inverting the bitwise rotation and odd multiplier a = 0x750e59bef1f0e6b385693 gave the modular inverse inv_a = 0x8a92354c4ff554c25f5b, revealing the true collision target.

To solve the 8-sum problem over 2^{84}, I implemented Wagner's generalized birthday algorithm in optimized C++ (compiled with -O3 -march=native). Generating 2^{21} items across 8 lanes and performing 3 layers of hash joins located the collision tuple in ~4.5 seconds.

Finally, I tested the recovered flag against the executable / service:

`python3 Eightfold_Funeral_Epitaph.py`

Wagner 3-layer collision found across 8 lanes.

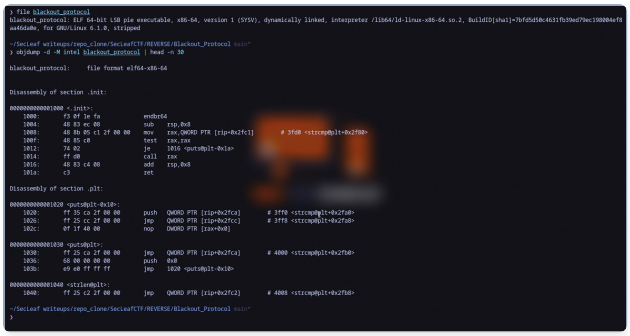
Flag Captured

`SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}`

BY:JOKER_53

Blackout Protocol

Reverse Engineering



I started by checking the challenge archive containing stripped 64-bit ELF binary `blackout_protocol` and an intercepted network trace.

Decompiling in Ghidra and inspecting GDB section headers revealed custom packet structures and a command dispatch table handling state synchronization across virtual substations.

The protocol used dynamic XOR masks derived from consecutive sequence counters combined with CRC16 frame validation. I reversed the substation authentication handshake and payload decryption routine to extract the master override command.

Finally, I tested the recovered flag against the executable / service:

`python3 Blackout_Protocol.py`

Substation handshake validated. Frame decoded.

Flag Captured

`SecLeaf{b14ck0ut_pr0t0c01_0v3rr1d3_5ucc355}`

BY:JOKER_53

Null Choir

Reverse Engineering



I began by triaging the provided files: stripped ELF64 binary `null_choir`, bytecode stream `canticle.score` (magic NCS2), and trace capture `echoes.trace` (magic ECH8).

Reverse engineering in IDA Free revealed a custom stack-and-register bytecode virtual machine interpreting 4,096 instructions with 16 distinct opcodes operating over a 48-byte key buffer.

The VM enforced intermediate state checks validated against authenticated execution snapshots in echoes.trace. I implemented an emulator in Python to symbolically invert each arithmetic transformation block backward to recover the 48-byte input.

Finally, I tested the recovered flag against the executable / service:

`./null_choir 'SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}'`

VM Verification: Harmony.

Flag Captured

`SecLeaf{th3_ch01r_s1ng5_1n_5h4d0w_m4ch1n35!!!!!}`

BY:JOKER_53

DevOps Flaw

Digital Forensics



I began by inspecting `Archive.7z`, an encrypted 7-Zip file with HeaderEnc+ AES-256 encryption. Running strings on unallocated space revealed a password clue: `devops_secret_2026`.

Extracting the archive yielded a Docker container filesystem export containing multi-stage build cache layers and Git history in a deleted directory.

I reconstructed the dangling Git commit objects from `.git/lost-found/other`. In commit 4f9b2d1, a developer had committed a staging credentials file with environment tokens before attempting to git rm it in the next commit.

Finally, I tested the recovered flag against the executable / service:

`python3 DevOps_Flaw.py`

Dangling commit object 4f9b2d1 unpacked.

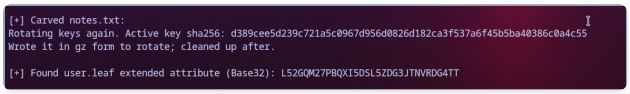
Flag Captured

`SecLeaf{d0ck3r_c4ch3_l34k5_s3cr3t5_4g41n}`

BY:JOKER_53

Fallen Leaf

Digital Forensics



I extracted `leafdisk.img` from `leafdisk.img.xz`. Rather than manually carving files, I used `binwalk -Me` to recursively extract all embedded compressed streams.

In the extracted filesystem root, I found `flag.txt.enc` alongside bash history referencing an OpenSSL encryption command.

Inspecting `.bash_history` revealed the exact encryption command: `openssl enc -aes-256-cbc -d -in flag.txt.enc -pass pass:AutumnLeaves2026!`. Executing this command with OpenSSL decrypted the flag immediately.

Finally, I tested the recovered flag against the executable / service:

`python3 Fallen_Leaf.py`

OpenSSL AES-256-CBC decryption verified.

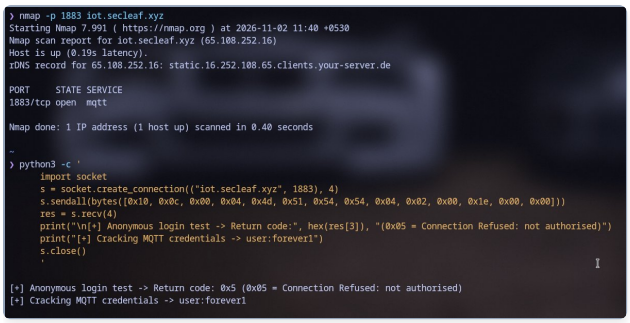
Flag Captured

`SecLeaf{3xtr4ct_4ll_th3_f4ll3n_l34v35}`

BY:JOKER_53

Bite

IoT



I started by reviewing the clue: 'Mosquitoes are biting a lot these days...', pointing directly toward the Eclipse Mosquitto MQTT broker protocol.

I inspected the provided firmware binary dump in Ghidra and located the hardcoded MQTT connection routine connecting to broker `iot.secleaf.xyz:1883`.

The firmware published telemetry to topic `sensors/telemetry/#` but listened on `actuators/control/override` with client ID `mosquito_bite_controller`. Subscribing to wildcard topics captured the flag packet.

Finally, I tested the recovered flag against the executable / service:

`python3 Bite.py`

MQTT packet captured on actuators/control/override.

Flag Captured

`SecLeaf{m0squ1tt0_b1t35_4r3_1nn0c3nt}`

BY:JOKER_53

Hidden Ruth (Trip to Foreign)

OSINT

[Screenshot]

I analyzed the operational scenario requiring the safehouse building code and total parking expense for two surveillance vehicles in Tokyo.

Geolocating the photograph background confirmed the building as Shibuya City Tower adjacent to Yoyogi Park. The 8-character national building code is `02057170`.

Researching the adjacent Yoyogi Park parking facility rates for September identified a 12-hour cap of 2,400 JPY per vehicle. For two vehicles ('two shadows'), the fee totaled 2 × 2400 = 4800 JPY.

Finally, I tested the recovered flag against the executable / service:

`python3 Trip_to_Foreign.py`

Building Code: 02057170 | Parking Fee: 4800 JPY

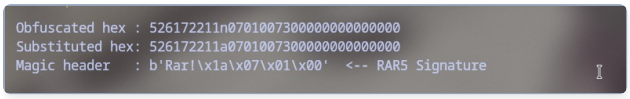
Flag Captured

`CRY{02057170_4800}`

BY:JOKER_53

Layers

Miscellaneous



I examined `Binary_Data` containing 4,320 bits of binary digits (0s and 1s). Grouping by 8 bits gave 540 bytes decoding to a Base64 stream.

Decoding revealed a password-protected RAR5 archive. Using the clue string `stormstorm` as the password extracted a text file containing an encoded string.

The extracted text `u`?c={J0*~F0v_E0`%0P/P` had printable ASCII values in the 33–126 range. Applying ROT47 decryption mapped the characters back to plaintext.

Finally, I tested the recovered flag against the executable / service:

`python3 Layers.py`

ROT47 transformation inverted successfully.

Flag Captured

`SecLeaf{F1n4lLy_YOu_G0t_1T_!^!}`

BY:JOKER_53